

SyBorx Messenger

Plataforma de Mensajería Empresarial y Comunidades

REPORTE TECNICO INTEGRAL

Documento: Reporte técnico de la aplicación web SyBorx Community

Version: 1.0

Fecha: 20 de agosto de 2026

Elaborado por: SyBorx Team

Estado: Núcleo funcional implementado y verificado

Indice

No se encontraron entradas de tabla de contenido.

1. Resumen Ejecutivo

SyBorx Community es la interfaz de usuario de la plataforma SyBorx Messenger: un sistema de mensajería empresarial y espacios de trabajo en equipo inspirado en herramientas como Discord y Slack. El proyecto combina un backend de alto rendimiento en NestJS con una base de datos relacional PostgreSQL contenedorizada, comunicación en tiempo real mediante WebSockets (Socket.io), autenticación basada en tokens JWT y control de acceso jerárquico por roles (RBAC).

El frontend es una aplicación de una sola página (SPA) construida con HTML, CSS y JavaScript nativo (sin frameworks), servida directamente por el propio backend NestJS. Su estética emplea el efecto de cristal esmerilado (liquid glass), tipografía Poppins y dos temas (claro y oscuro).

El núcleo del sistema ha sido implementado y verificado de extremo a extremo: registro e inicio de sesión, directorio de usuarios, comunidades con canales temáticos, chats directos 1 a 1, mensajes en tiempo real en canales y conversaciones, presencia online/offline, indicador de escritura, subida de archivos adjuntos y cierre de sesión. Este documento describe la arquitectura completa, el modelo de datos, la API, los mecanismos de seguridad y el estado actual, y propone un plan de mejoras para continuar su evolución.

2. Contexto y Objetivos

2.1 Contexto

Las organizaciones necesitan un canal interno de comunicación en tiempo real que centralice conversaciones de equipo, anuncios y trabajo por proyectos. SyBorx Messenger cubre esa necesidad mediante comunidades (servidores) con canales temáticos, mensajería directa, presencia en línea y roles de acceso diferenciados.

2.2 Objetivos del proyecto

Proporcionar comunicación en tiempo real entre equipos mediante WebSockets.

Organizar la comunicación en comunidades y canales temáticos (tipo #general, #dev-backend, #anuncios).

Permitir chats directos privados 1 a 1 con historial persistente.

Aplicar control de acceso jerárquico por roles globales y por comunidad (RBAC).

Soportar envío de imágenes, audios y documentos adjuntos.

Exponer una API documentada con Swagger/OpenAPI para integraciones futuras.

Entregar una interfaz web moderna, responsiva y con dos temas de color.

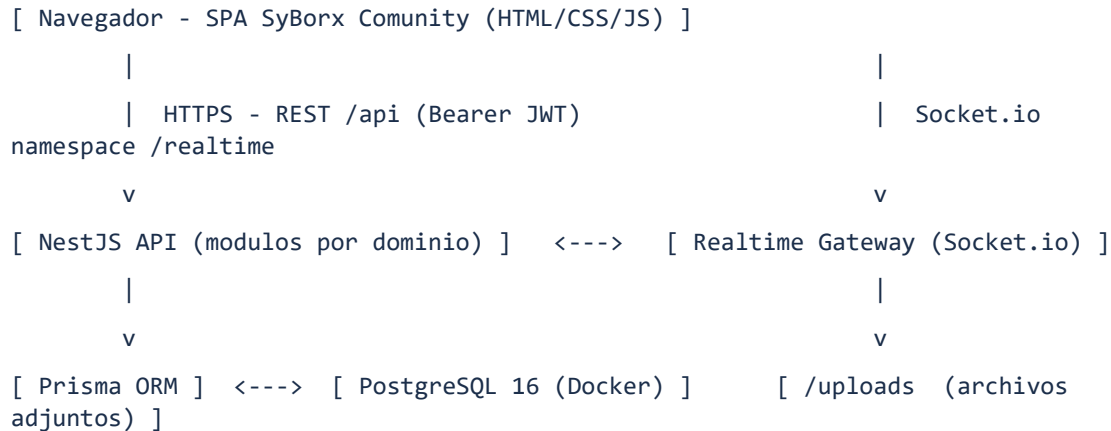
2.3 Alcance

El alcance actual abarca el backend completo (autenticación, usuarios, comunidades, canales, chats directos, mensajes, archivos, administración, salud) y el frontend SPA

funcional conectado en tiempo real. Quedan para fases posteriores: videollamadas WebRTC, asistente con IA, notificaciones push y almacenamiento en la nube.

3. Arquitectura General del Sistema

3.1 Vision de alto nivel



3.2 Frontend (capa de presentacion)

SPA en JavaScript nativo servida por NestJS desde la carpeta public/. Mantiene el token JWT en el navegador (localStorage), consume la API REST con cabecera Authorization y mantiene una conexion WebSocket permanente para mensajes, presencia y escritura. Incluye cliente Socket.io vendored (sin CDN).

3.3 Backend (capa de servicios)

NestJS 10 con arquitectura modular por dominio: cada modulo agrupa controller (HTTP), service (logica de negocio) y DTOs con validacion. Los guards se registran de forma global (autenticacion JWT) y especifica (roles de administracion), con un pipe global de validacion y un filtro global de excepciones.

3.4 Base de datos

PostgreSQL 16 ejecutado en contenedor Docker, gestionado con Prisma ORM. El esquema define 11 entidades relacionadas con claves UUID, timestamps y reglas de borrado en cascada, ademas de seis enumerados de negocio.

3.5 Tiempo real

Gateway Socket.io en el namespace /realtime con autenticacion JWT, salas por canal (channel_<id>) y por conversacion (direct_chat_<id>), emision de mensajes nuevos, presencia online/offline y eventos de escritura.

4. Stack Tecnologico

Capa	Tecnologia	Version	Proposito
------	------------	---------	-----------

Backend	NestJS (Node.js + TypeScript)	10 / 5.4	Framework modular y guardas globales
ORM	Prisma	5.14	Migraciones y acceso tipado a datos
Base de datos	PostgreSQL	16	Persistencia relacional
Tiempo real	Socket.io	4.7 / 4.8	WebSockets con salas y presencia
Autenticacion	Passport + JWT + Bcrypt	0.7 / 10	Access + refresh tokens
Documentacion	Swagger / OpenAPI	7.3	UI interactiva en /api/docs
Archivos	Multer	1.4	Subida multipart de adjuntos
Frontend	HTML5 + CSS3 + JS vanilla	-	SPA liquid glass, dos temas
Infraestructura	Docker / Docker Compose	24+	Entornos dev y produccion
Salud	NestJS Terminus	10.2	Healthcheck en /api/health

5. Backend: Modulos y API REST

5.1 Modulos

Modulo	Responsabilidad
auth	Registro, login, refresh, logout y perfil actual (/auth/me)
users	Directorio de integrantes, perfil y presencia
admin	Gestion de usuarios, roles y auditoria (ADMIN/SUPERADMIN)
communities	Comunidades o servidores y sus miembros
channels	Canales tematicos dentro de una comunidad
direct-chats	Conversaciones directas 1 a 1
messages	Mensajes de canal y DM, edicion, borrado y reacciones
files	Subida de adjuntos via multipart
realtime	Gateway WebSocket /realtime con presencia
health	Healthcheck de la aplicacion
database	Servicio global Prisma
common	Guards, filtro HTTP, interceptor de logging y paginacion

5.2 Autenticacion (publico)

Metodo	Ruta	Descripcion
POST	/api/auth/register	Registra usuario, crea tokens y auto-une a la comunidad publica
POST	/api/auth/login	Inicia sesion con correo o usuario y contrasena
POST	/api/auth/refresh	Renueva el par de tokens con el refresh token
POST	/api/auth/logout	Cierra la sesion y revoca los refresh tokens
GET	/api/auth/me	Devuelve el usuario autenticado actual

5.3 Usuarios

Metodo	Ruta	Descripcion
GET	/api/users/directory	Directorio de integrantes para chats directos
GET	/api/users/:id	Detalle publico de un usuario
PATCH	/api/users/profile	Actualiza nombre, avatar o biografia propios
PATCH	/api/users/presence	Actualiza el estado de presencia del usuario

5.4 Administracion (ADMIN / SUPERADMIN)

Metodo	Ruta	Descripcion
GET	/api/admin/users	Listado de usuarios con filtros y paginacion
PATCH	/api/admin/users/:id/role	Cambia el rol global de un usuario
PATCH	/api/admin/users/:id/status	Activa o desactiva una cuenta
GET	/api/admin/audit-logs	Registros de auditoria de acciones administrativas

5.5 Comunidades y canales

Metodo	Ruta	Descripcion
POST	/api/communities	Crea una comunidad (segun permisos RBAC)
GET	/api/communities/my	Comunidades del usuario autenticado
GET	/api/communities/:id	Detalle de una comunidad con sus canales

POST	/api/communities/:id/members	Agrega un miembro a la comunidad
POST	/api/communities/:id/join	El usuario se une a una comunidad publica
DELETE	/api/communities/:id/leave	El usuario abandona la comunidad
POST	/api/communities/:communityId/channels	Crea un canal en la comunidad
GET	/api/channels/:id	Detalle de un canal
DELETE	/api/channels/:id	Elimina un canal (administradores)

5.6 Chats directos y mensajes

Metodo	Ruta	Descripcion
POST	/api/direct-chats	Crea u obtiene el chat directo con otro usuario
GET	/api/direct-chats	Lista de conversaciones directas del usuario
GET	/api/direct-chats/:id	Detalle de una conversacion directa
POST	/api/messages	Crea un mensaje (canal o DM) y lo transmite en vivo
GET	/api/channels/:channelId/messages	Historial paginado de un canal
GET	/api/direct-chats/:chatId/messages	Historial paginado de un DM
PATCH	/api/messages/:id	Edita un mensaje propio (marca isEdited)
DELETE	/api/messages/:id	Elimina un mensaje (autor o admin de comunidad)
POST	/api/messages/:id/reactions	Agrega o quita una reaccion (toggle)

5.7 Archivos y salud

Metodo	Ruta	Descripcion
POST	/api/files/upload	Sube un adjunto (multipart) y devuelve la URL publica
GET	/api/health	Healthcheck del servicio (publico)

6. Tiempo Real (WebSockets)

El gateway escucha en el namespace /realtime con CORS abierto y autenticacion por JWT (enviado en handshake.auth.token o en la cabecera Authorization). Al conectarse marca al usuario como ONLINE y emite user:presence; al desconectarse lo vuelve OFFLINE. Los mensajes nuevos se emiten a la sala del canal o de la conversacion correspondiente.

6.1 Eventos del cliente al servidor

Evento	Payload	Efecto
channel:join	{ channelId }	Une el socket a la sala channel_<id>
channel:leave	{ channelId }	Saca el socket de la sala channel_<id>
direct_chat:join	{ chatId }	Une el socket a la sala direct_chat_<id>
typing:start	{ channelId directChatId }	Avisa que el usuario esta escribiendo
typing:stop	{ channelId directChatId }	Avisa que el usuario dejo de escribir

6.2 Eventos del servidor al cliente

Evento	Contenido
message:new	Mensaje completo con remitente, adjuntos y reacciones
user:presence	{ userId, status } ONLINE u OFFLINE
user:typing	{ user, isTyping, channelId/directChatId }

7. Modelo de Datos

7.1 Entidades

Entidad	Descripción	Relaciones clave
User	Cuenta del empleado o prueba	RefreshTokens, membresias, mensajes, reacciones, auditoria
RefreshToken	Sesion de refresh (token hashado)	Pertenece a User (borrado en cascada)
Community	Comunidad o servidor	Owner User, Channels, CommunityMembers
CommunityMember	Membresia con rol de comunidad	Community + User (unico por par)
Channel	Canal tematico de una comunidad	Community (cascada), Messages
DirectChat	Conversacion 1 a 1	DirectChatMembers, Messages
DirectChatMember	Participante del DM con ultima lectura	DirectChat + User (unico por par)

Message	Mensaje de canal o DM	Sender, Channel/DirectChat, parentMessage (hilos)
Attachment	Archivo adjunto del mensaje	Message (cascada)
MessageReaction	Reaccion emoji de un usuario	Message + User (unico por terna)
AuditLog	Bitacora de acciones administrativas	User (borrado nulo)

7.2 Enumerados

Enumerado	Valores
GlobalRole	SUPERADMIN, ADMIN, TECH_LEAD, DEVELOPER, GUEST
UserStatus	ONLINE, IDLE, DND, OFFLINE
AuthProvider	LOCAL, GOOGLE
CommunityRole	COMMUNITY_OWNER, COMMUNITY_ADMIN, COMMUNITY_MODERATOR, COMMUNITY_MEMBER
ChannelType	TEXT, VOICE_PREP, ANNOUNCEMENT
MessageType	TEXT, IMAGE, FILE, AUDIO, SYSTEM, AI_BOT

7.3 Notas de diseno

Identificadores UUID v4 generados por Prisma; timestamps created_at y updated_at en casi todas las entidades.

Borrados en cascada para contenido dependiente (mensajes de canal, adjuntos, reacciones).

VOICE_PREP y AI_BOT preparan el esquema para videollamadas WebRTC y chatbot con IA sin migraciones destructivas.

ATTACHMENTS guarda metadatos desacoplados (file_url, stored_name, mime_type) listos para migrar a S3/MinIO.

8. Control de Acceso (RBAC)

8.1 Matriz de roles globales

Rol global	Jerarquia	Crear comunidades	Gestionar usuarios	Auditoria
SUPERADMIN	5	Si	Si (total)	Si
ADMIN	4	Si	Si (roles inferiores)	Si
TECH_LEAD	3	Si	No	No
DEVELOPER	2	Si	No	No
GUEST	1	No	No	No

8.2 Roles de comunidad

Dentro de cada comunidad los miembros tienen un rol propio: `COMMUNITY_OWNER`, `COMMUNITY_ADMIN`, `COMMUNITY_MODERATOR` y `COMMUNITY_MEMBER`. Los administradores de la comunidad pueden crear y eliminar canales y moderar mensajes.

8.3 Implementacion

`JwtAuthGuard` registrado de forma global: toda ruta exige token salvo las marcadas con `@Public` (login, registro, refresh, health).

`RolesGuard` + `@Roles` para las rutas de administracion (solo `ADMIN` y `SUPERADMIN`).

El primer usuario registrado en la base de datos recibe automaticamente el rol `SUPERADMIN`; el resto, `DEVELOPER`.

Cada servicio valida pertenencia (membresia a la comunidad o participacion en el DM) antes de permitir leer o escribir mensajes.

Al registrarse, el usuario se une automaticamente a la primera comunidad publica como `COMMUNITY_MEMBER`.

9. Frontend: Interfaz de Usuario

9.1 Pantalla de autenticacion

Tarjeta flotante con efecto liquid glass y dos pestanas: Iniciar sesion y Crear cuenta. El formulario de registro solicita nombre, correo corporativo o Gmail, usuario, contrasena y biografia opcional. Incluye boton de cambio de tema y una pista con las cuentas de prueba.

9.2 Aplicacion principal

Sidebar (30%): tarjeta de perfil (avatar, nombre y rol; al hacer clic abre el modal Mi cuenta), boton de cambio de tema, boton Cerrar sesion, selector Comunidad / Grupos y la lista de integrantes o canales.

Panel central (70%): cabecera del chat con nombre del canal (`#nombre`) o del contacto, boton de miembros en canales, lista de mensajes con globos propios y ajenos, reacciones, adjuntos y marca de editado, y el compositor con area de texto, adjuntar archivo y enviar.

Modal de integrantes: muestra los miembros de la comunidad con su rol.

Modal Mi cuenta: datos del usuario autenticado y opcion de cerrar sesion.

9.3 Sistema de disenio

Efecto liquid glass (fondos translucidos con blur y saturacion) sobre gradientes de fondo.

Tipografia Poppins cargada desde Google Fonts con fallback local.

Tema claro (crema + azul marino #1B2A4A) y tema oscuro, con persistencia en localStorage.

Indicadores de presencia (punto verde/offline) en avatares e integrantes, escritura en vivo y reacciones.

9.4 Integración con el backend

API REST con cabecera Authorization: Bearer <accessToken>; token y tema persistidos en localStorage.

Cliente Socket.io vendored en public/vendor/socket.io.min.js (sin dependencia de CDN).

El frontend se sirve desde NestJS con fallback SPA que excluye /api, /uploads y /socket.io para no interferir con el backend.

10. Flujos de Usuario Principales

10.1 Registro

1. El usuario llena el formulario Crear cuenta. 2. El backend valida la política de dominios y la unicidad de correo/usuario. 3. Cifra la contraseña con Bcrypt y crea la cuenta (primer usuario = SUPERADMIN). 4. Lo une a la comunidad pública. 5. Devuelve los tokens JWT y se muestra la aplicación.

10.2 Inicio de sesión

1. El usuario envía correo o usuario + contraseña. 2. El backend verifica credenciales y estado activo. 3. Genera access (1 día) y refresh (7 días, almacenado hasheado). 4. El frontend guarda el token y conecta el WebSocket, quedando ONLINE.

10.3 Chats de grupo

1. El usuario abre la pestaña Grupos y selecciona un canal. 2. La cabecera muestra el nombre del canal (#general) y la comunidad. 3. Se cargan los mensajes paginados y el socket se une a la sala del canal. 4. Los mensajes nuevos llegan en vivo a todos los miembros.

10.4 Chats directos

1. En la pestaña Comunidad el usuario elige un integrante del directorio. 2. Se crea u obtiene el DM. 3. Ambos participantes se unen a la sala del chat y reciben mensajes en tiempo real.

10.5 Adjuntos

1. El usuario adjunta un archivo en el compositor. 2. Se sube a /api/files/upload y se guarda en /uploads. 3. El mensaje se crea con el adjunto y se emite por WebSocket con su URL pública.

10.6 Presencia y escritura

1. Al conectarse, el gateway publica user:presence ONLINE. 2. Mientras el usuario escribe, se emite typing:start y la cabecera muestra quien esta escribiendo. 3. Al enviar o detenerse, typing:stop restaura el estado.

10.7 Cerrar sesion

1. El usuario pulsa Cerrar sesion. 2. El backend revoca los refresh tokens. 3. El frontend elimina el token y muestra de nuevo el formulario de autenticacion.

11. Seguridad

Contraseñas cifradas con Bcrypt (cost 10); nunca se almacenan en texto plano.

Refresh tokens hasheados en base de datos (Bcrypt cost 6) con expiracion de 7 dias; revocacion al cerrar sesion.

Dos secretos JWT independientes (access y refresh) configurables por variables de entorno.

JwtAuthGuard global con decorador @Public para rutas abiertas; RolesGuard para rutas administrativas.

ValidationPipe global con whitelist y forbidNonWhitelisted: se descarta o rechaza cualquier campo no declarado en los DTO.

Politica de correos: solo dominios corporativos (@syborx.com) o Gmail cuando se habilita el modo de pruebas.

Chequeos de autorizacion en cada servicio (membresia de comunidad / participacion en DM) para mensajes y canales.

El fallback SPA excluye /api, /uploads y /socket.io para no enmascarar las rutas del backend.

CORS configurable por variable de entorno; CORS habilitado para el cliente web.

Filtro global de excepciones y registro (logging interceptor) de las peticiones.

12. Despliegue y Entorno

12.1 Entornos

docker-compose.yml (dev): levanta PostgreSQL 16 y pgAdmin; el backend corre con npm run start:dev.

docker-compose.prod.yml: imagen multi-stage del backend, PostgreSQL con volúmenes persistentes y ejecución automática de migraciones Prisma.

Dockerfile multi-stage para compilar y ejecutar en producción con Node.js 20.

12.2 Variables de entorno principales

Variable	Descripcion	Valor por defecto
----------	-------------	-------------------

DATABASE_URL	Cadena de conexion a PostgreSQL	postgresql://...:5432/syborx
JWT_SECRET	Secreto del access token	syborx_jwt_secret_dev
JWT_REFRESH_SECRET	Secreto del refresh token	syborx_refresh_secret_dev
JWT_EXPIRES_IN / JWT_REFRESH_EXPIRES_IN	Vigencia de los tokens	1d / 7d
APP_URL	URL publica base (para archivos)	http://localhost:3000
CORS_ORIGIN	Origenes permitidos (separados por coma)	*
ALLOWED_CORP_DOMAIN	Dominio corporativo permitido	syborx.com
ALLOW_GMAIL_TESTING	Permite Gmail en modo pruebas	false
PORT / API_PREFIX	Puerto y prefijo de la API	3000 / api
MAX_FILE_SIZE_MB	Tamano maximo de adjuntos	25
UPLOAD_DEST	Directorio de archivos subidos	./uploads

12.3 Entorno local actual

En la maquina de desarrollo el PostgreSQL nativo de Windows ocupa el puerto 5432, por lo que el contenedor Docker se publica en el puerto 5433. La aplicacion se sirve en <http://localhost:3000> con Swagger en </api/docs> y healthcheck en </api/health>.

13. Estado Actual del Proyecto

13.1 Implementado y verificado de extremo a extremo

Backend compilado y servidor corriendo en el puerto 3000.

Verificados en ejecucion: registro con auto-join a comunidad publica, login, </auth/me>, directorio, comunidades, chats directos, historial de mensajes y subida de archivos.

Tiempo real probado con dos clientes simultaneos: mensajes en canal y en DM, presencia y escritura.

Frontend SPA servido por NestJS con fallback correcto y cliente Socket.io vendored.

13.2 Ajustes recientes de interfaz

La cabecera de los chats de grupo muestra ahora el nombre real del canal (por ejemplo #general) en lugar de un texto generico.

El boton Cerrar sesion se ubico junto al cambio de tema con el mismo estilo redondeado; el menu desplegable previo se sustituyo por acceso directo al modal Mi cuenta desde el avatar.

Los campos de los formularios (email, contrasena, busqueda) comparten el mismo diseno estilizado.

13.3 Notas

Quedo creada la cuenta de prueba jorge.prueba@syborx.com (DEVELOPER) utilizada para verificar el flujo de registro. Los mensajes y archivos de prueba fueron eliminados.

14. Areas de Mejora y Proximos Pasos

Prioridad	Area	Mejora propuesta
Alta	Seguridad	Mover el access token a cookies httpOnly con rotacion y revocacion de refresh tokens ante reuso
Alta	Seguridad	Limitar peticiones (rate limiting) en login y registro; cabeceras Helmet y proteccion CSRF
Alta	Calidad	Suite de pruebas unitarias (Jest) y de integracion (e2e) con cobertura de los servicios clave
Media	Mensajeria	Busqueda global de mensajes e integrantes
Media	Mensajeria	Interfaz para editar, eliminar y fijar mensajes; hilos de respuesta visibles
Media	UX	Notificaciones de mensajes nuevos y sonidos; leído/no leído (check azul) por DM
Media	Admin	Panel de administracion web para gestionar usuarios, roles y auditoria
Media	Rendimiento	Carga incremental (infinite scroll) del historial y paginacion por cursor
Baja	Tiempo real	Videollamadas y salas de voz WebRTC (el esquema ya contempla VOICE_PREP)
Baja	IA	Asistente con chatbot y hilos de contexto (MessageType AI_BOT ya definido)
Baja	Almacenamiento	Migrar adjuntos a S3/MinIO mediante variables de entorno
Baja	Plataforma	Internacionalizacion (i18n), accesibilidad (ARIA) y version movil responsiva

15. Conclusiones

SyBorx Messenger cuenta con un backend empresarial robusto y modular sobre NestJS, PostgreSQL y Docker, con autenticación JWT, RBAC jerárquico, WebSockets en tiempo real, subida de archivos y API documentada con Swagger.

El frontend SyBorx Comunity ofrece una experiencia visual cuidada (liquid glass, dos temas) y completamente funcional, integrada de extremo a extremo con la API y el tiempo real.

La base de datos y los enumerados están preparados para evolucionar hacia llamadas, IA y almacenamiento en la nube sin romper el esquema. Se recomienda priorizar las mejoras de seguridad y la automatización de pruebas para consolidar el sistema antes de su puesta en producción.

Anexo A. Cuentas de Prueba (Seed)

Rol	Correo	Contraseña
SUPERADMIN (CEO)	ceo@syborx.com	Password123!
ADMIN (SysAdmin)	admin@syborx.com	Password123!
TECH_LEAD	techlead@syborx.com	Password123!
DEVELOPER	dev@syborx.com	Password123!
GUEST (QA Gmail)	tester.syborx@gmail.com	Password123!

Anexo B. Documentación y Recursos

README.md - descripción general e inicio rápido.

docs/DATABASE_ARCHITECTURE.md - ERD en Mermaid y diccionario de datos.

docs/DEPLOYMENT_GUIDE.md - despliegue en desarrollo y producción.

Swagger UI interactiva: <http://localhost:3000/api/docs>

Healthcheck: <http://localhost:3000/api/health>

Realtime Gateway: namespace /realtime (Socket.io)